

Résumé — Cours Go (langage Go)

Prise de notes — 29/05/2026 — M. Raphaël Canetti

1.3 Les types de base en Go

Go est un langage à **typage statique** : chaque variable a un type connu à la compilation. Pas de surprise à l'exécution.

Types numériques entiers

```
var a int    = 42          // int : 32 ou 64 bits selon la plateforme (recommandé)
var b int8   = 127         // -128 à 127
var c int16  = 32767       // -32768 à 32767
var d int32  = 2147483647
var e int64  = 9223372036854775807
var u uint   = 42          // entier non signé
var u8 uint8 = 255         // 0 à 255 = byte
```

Types numériques flottants

```
var f float32 = 3.14          // précision ~7 chiffres
var g float64 = 3.14159265358979 // précision ~15 chiffres (recommandé)
```

Types fondamentaux

```
var h string = "Bonjour Go!" // UTF-8 natif (emojis, accents, kanji...)
var i bool   = true
```

Types alias

```
var j byte = 65 // byte = uint8 → données binaires, ASCII
var k rune = 'é' // rune = int32 → caractère Unicode
```

Affichage avec printf

```
fmt.Printf("Entier: %d, Flottant: %.2f, Chaîne: %s\n", a, g, h)
fmt.Printf("Type: %T, Valeur: %v\n", g, g) // %T = type, %v = valeur
```

Avantage Go : pas de conversion implicite. `var x int = 3.14` ne compile pas. C'est une fonctionnalité, pas un bug : ça évite des bugs subtils de précision qui coûtent des millions en production (cf. bug Ariane 5 : overflow int16→int64).

Les valeurs zéro — une sécurité unique à Go

En Go, toute variable déclarée est automatiquement initialisée à une **valeur zéro**. Fini les bugs dus aux variables non initialisées.

```
var monEntier    int        // = 0      (pas de "undefined" ni de "null" !)  
var monFlottant float64    // = 0.0  
var maChaine     string    // = ""     (chaîne vide)  
var monBool      bool      // = false  
var monPointeur *int       // = nil    (seul cas de nil en Go)  
var monSlice     []int     // = nil  
var maMethode    func()    // = nil
```

```
// Une struct aussi est initialisée à zéro !  
type Personne struct { Nom string; Age int }  
var p Personne // Personne{Nom: "", Age: 0}
```

```
// Utile pour des patterns idiomatiques  
var compteur int  
for i := 0; i < 10; i++ {  
    compteur++ // pas besoin de := 0, c'est déjà 0  
}
```

Slices (tranches)

- Un **array** a une taille fixe ; un **slice** est une vue dynamique et flexible sur les éléments d'un array. En pratique les slices sont bien plus courants.
- Le type `[]T` est un slice d'éléments de type `T`.
- On forme un slice avec deux indices (bornes basse et haute) séparés par `:` → `a[low : high]`.
- C'est un **intervalle semi-ouvert** : inclut le premier élément, **exclut** le dernier.

```
package main  
  
import "fmt"  
  
func main() {  
    primes := [6]int{2, 3, 5, 7, 11, 13}  
  
    var s []int = primes[1:4] // éléments d'indice 1 à 3 → [3 5 7]  
    fmt.Println(s)  
}
```

1.4 Variables et constantes

Les 3 façons de déclarer une variable

```
// 1. var avec type et valeur explicites (n'importe où dans le code)
var prenom string = "Alice"
var age    int    = 30

// 2. var avec inférence de type (Go déduit le type)
var ville = "Paris"    // Go sait que c'est un string
var prix  = 19.99      // Go sait que c'est un float64

// 3. Déclaration courte := (UNIQUEMENT dans les fonctions !)
//    C'est la façon la plus courante en pratique
nom      := "Bob"
score    := 42
```

Astuces

```
// Plusieurs variables à la fois
x, y := 10, 20
a, b := "hello", true

// Échange de valeurs – Go est élégant ici !
x, y = y, x // x=20, y=10 (pas besoin de variable temporaire !)
```



```
// Déclarations groupées
var (
    serveur string = "localhost"
    port    int    = 8080
    actif   bool    = true
)
```

Constantes & iota

```
// iota avec calcul – tailles de stockage
const (
    Octet = 1
    Ko     = 1 << (iota + 1) // 1024
    Mo     // 1048576
    Go     // 1073741824
    To     // 1099511627776
)

// Exemple réel : jours de la semaine
type Jour int
const (
    Lundi Jour = iota + 1 // 1 (pas 0)
    Mardi           // 2
    Mercredi        // 3
    Jeudi           // 4
    Vendredi        // 5
    Samedi          // 6
    Dimanche        // 7
)
```

Exercice noté 1 – Calculateur d'IMC et de catégorie (15 min)

L'IMC (Indice de Masse Corporelle) est utilisé en médecine. Premier exercice mêlant types, variables et constantes.

1. Déclarer deux variables `float64` : poids (ex. 70.5 kg) et taille (ex. 1.75 m)
2. Créer 4 constantes : `IMCMaigre=18.5` , `IMCNormal=25.0` , `IMCSurpoids=30.0`
3. Calculer l'IMC = `poids / (taille * taille)`
4. Afficher l'IMC avec 2 décimales (`fmt.Printf` avec `%.2f`)
5. Afficher la catégorie : Maigre / Normal / Surpoids / Obésité
6. **Bonus** : déclarer une constante `Nom` avec son prénom et personnaliser l'affichage

Rendu : fichier `imc.go` sur GitHub.

1.5 Les fonctions

Retour multiple — le pattern idiomatique (résultat, erreur)

```
func diviser(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zéro : impossible de diviser %v par 0", a)
    }
    return a / b, nil // nil = pas d'erreur
}
```

```
// Utilisation – toujours vérifier l'erreur !
resultat, err := diviser(10, 3)
if err != nil {
    fmt.Println("Erreur:", err)
    return
}
fmt.Printf("10 / 3 = %.4f\n", resultat) // 3.3333
```

```
// Ignorer une valeur de retour avec _
resultat2, _ := diviser(20, 4) // on ignore l'erreur si on est sûr
```

Retours nommés / naked return (pour fonctions courtes et documentaires)

```
func statistiques(nums []float64) (min, max, moyenne float64) {
    if len(nums) == 0 {
        return // naked return : retourne 0, 0, 0 (valeurs zéro)
    }
    min, max = nums[0], nums[0]
    var somme float64
    for _, n := range nums {
        if n < min { min = n }
        if n > max { max = n }
        somme += n
    }
    moyenne = somme / float64(len(nums))
    return // naked return : retourne min, max, moyenne
}
```

1.6 Le contrôle de flux — `for` : LA seule boucle en Go

`for` remplace `while`, `do-while`, `for`, `foreach`.

```
// for classique (comme en C)
for i := 0; i < 5; i++ {
    fmt.Println(i) // 0 1 2 3 4
}

// for 'while' – condition seule
n := 1
for n < 100 {
    n *= 2
}
fmt.Println(n) // 128

// for infini – break pour sortir
for {
    fmt.Print("Entrez un nombre (0 pour quitter): ")
    var x int
    fmt.Scan(&x)
    if x == 0 { break }
    fmt.Println(x * x)
}
```

```
// for range – itération sur collections
fruits := []string{"pomme", "banane", "cerise"}
for index, valeur := range fruits {
    fmt.Printf("[%d] %s\n", index, valeur)
}

// Ignorer l'index
for _, fruit := range fruits {
    fmt.Println(fruit)
}

// Sur une string : itère sur les runes (Unicode !)
for i, c := range "Héllo" {
    fmt.Printf("pos=%d, char=%c\n", i, c)
    // 'é' = 2 bytes en UTF-8, donc le prochain index est 3 !
}
```

Exercice noté 2 – Calculatrice avec closures et validation (20 min)

Une calculatrice CLI qui démontre les fonctions, retours multiples et closures.

1. Créer une fonction `operer(a, b float64, op string) (float64, error)` qui supporte `+` `-` `*` `/` `-` retourne une erreur pour `/` par 0 ou opération inconnue
2. Créer une fonction `creerOperation(op string) func(float64, float64) float64` qui retourne une closure pour chaque opération
3. Dans `main()`, faire une boucle `while true` qui :
4. Lit deux nombres et une opération depuis l'entrée standard (`fmt.Scan`)
5. Affiche le résultat ou l'erreur

6. Sort si l'opération est `quit`

7. Tester avec : `10 5 +` → 15 | `8 0 /` → erreur | `3 4 *` → 12

Rendu : fichier `calculatrice.go` sur GitHub.

Indices : `switch op { case "+": ... } | fmt.Scan(&a, &b, &op) | for { if op == "quit" { break } }`

2. Les structs — types personnalisés

```
type Produit struct {
    Nom      string
    Prix     float64
    Stock    int
    Actif    bool
    Categories []string // un slice dans une struct, c'est possible !
}
```

3 façons d'initialiser une struct

```
// 1. Positionnelle (fragile si on ajoute des champs !)
p1 := Produit{"iPhone 15", 999.99, 50, true, []string{"high-tech"}}

// 2. Nommée (RECOMMANDÉE — robuste aux changements)
p2 := Produit{
    Nom:      "MacBook Pro",
    Prix:     2499.99,
    Stock:    10,
    Actif:    true,
    Categories: []string{"high-tech", "ordinateur"},
}

// 3. Valeur zéro puis affectation
var p3 Produit
p3.Nom = "AirPods"
p3.Prix = 199.99
```

3.2 Méthodes — donner des comportements aux structs

Value receiver : travaille sur une COPIE → ne modifie pas l'original. Nommage : 1-2 lettres du type. **Pointer receiver** `*T` : travaille sur le VRAI objet → MODIFIE l'original. À utiliser pour modifier, ou quand la struct est grande.

```
// Value receiver : lecture seule
func (p Produit) Description() string {
    statut := "indisponible"
    if p.Actif && p.Stock > 0 {
        statut = "disponible"
    }
    return fmt.Sprintf("%s - %.2f€ (%S, stock: %d)", p.Nom, p.Prix, statut, p.Stock)
}

func (p Produit) PrixTTC() float64 {
    return p.Prix * 1.20 // TVA 20%
}

// Pointer receiver : modifie l'original
func (p *Produit) AppliquerReduction(pct float64) {
    if pct < 0 || pct > 100 {
        return // validation
    }
    p.Prix = p.Prix * (1 - pct/100)
}

func (p *Produit) Vendre(quantite int) error {
    if quantite > p.Stock {
        return fmt.Errorf("stock insuffisant : %d demandé, %d disponible", quantite, p.Stock)
    }
    p.Stock -= quantite
    return nil
}
```

Go gère automatiquement `& et *` : `prod.AppliquerReduction(10)` fait `&prod` tout seul.

3.3 Embedding — composition (l'« héritage » de Go)

On embarque un type **sans nom de champ** : toutes ses méthodes sont **promues** vers le type englobant.


```

type Adresse struct {
    Rue, Ville, CodePostal, Pays string
}
func (a Adresse) Format() string {
    return fmt.Sprintf("%s, %s %s, %s", a.Rue, a.CodePostal, a.Ville, a.Pays)
}

type Personne struct {
    Prenom, Nom string
    Age        int
}
func (p Personne) NomComple() string { return p.Prenom + " " + p.Nom }

// Client CONTIENT une Personne et une Adresse (embedding)
type Client struct {
    Personne    // embedding : méthodes de Personne promues
    Adresse     // embedding : méthodes d'Adresse promues
    Email       string
    NumeroClient int
}

c := Client{
    Personne:    Personne{Prenom: "Alice", Nom: "Martin", Age: 30},
    Adresse:     Adresse{Rue: "12 rue de la Paix", Ville: "Paris", CodePostal: "75001", Pays:
"France"},
    Email:       "alice@example.com",
    NumeroClient: 1001,
}
fmt.Println(c.NomComple()) // promotion de Personne.NomComple()

```

3.4 Struct Tags — métadonnées (JSON et plus)

Les tags sont des chaînes entre **backticks** après chaque champ.

```

type Utilisateur struct {
    ID      int    `json:"id"`           // sérialisé comme "id"
    Prenom  string `json:"prenom"`
    Email   string `json:"email,omitempty"` // omis si vide ("" )
    Age     int    `json:"age,omitempty"`  // omis si 0
    Interne string `json:"- "`           // JAMAIS sérialisé
    MDP     string `json:"- "`           // mot de passe : jamais !
}

```

```
import "encoding/json"

// Marshal : struct → JSON
u := Utilisateur{ID: 1, Prenom: "Alice", Email: "alice@go.dev", Age: 30}
data, _ := json.Marshal(u)
fmt.Println(string(data))
// {"id":1,"prenom":"Alice","email":"alice@go.dev","age":30}

// Unmarshal : JSON → struct
jsonStr := `{"id":2,"prenom":"Bob"}`
var u2 Utilisateur
json.Unmarshal([]byte(jsonStr), &u2)
fmt.Println(u2.Prenom) // Bob
```

Exercice noté 3 — Système de contacts : Personne, Employé, Étudiant (20 min)

Modéliser un annuaire d'entreprise avec différents types de contacts.

1. Créer un struct `Personne { Prenom, Nom string; Age int; Email string }` avec les méthodes `NomComplet() string` et `Presentation() string`
2. Créer `Adresse { Rue, Ville, CodePostal string }` avec `Format() string`
3. Créer `Employe` en **embeddant** `Personne` et `Adresse` + champs `Poste string`, `Salaire float64`
4. Méthode `FicheEmploye() string` qui affiche toutes les infos
5. Méthode `AugmenterSalaire(pct float64)` qui modifie le salaire (*pointer receiver*)
6. Créer `Etudiant` en embeddant `Personne` + champs `Promo string`, `Moyenne float64`
7. Méthode `MentionObtenue() string` (TB / B / AB / P selon la moyenne)
8. Dans `main()`, créer 2 employés et 2 étudiants, afficher leurs fiches

Rendu : fichier `contacts.go` sur GitHub.

Indices : (e *Employe) pour modifier le salaire | `switch { case moy >= 16: ... } | fmt.Sprintf` pour construire des chaînes

5.1 Packages & visibilité

En Go, la visibilité est déterminée par la **CASSE du premier caractère** du nom. Pas de mots-clés `public / private / protected`. Simple et universel. - **Majuscule = exporté** (accessible depuis n'importe où) - **minuscule = non exporté** (privé au package)

```
// fichier : boutique/produit.go
package boutique // nom du package (minuscule, sans underscore)

import (
    "fmt"
    "errors"
)

// EXPORTÉ – commence par une majuscule
type Produit struct {
    Nom    string
    Prix   float64 // exporté
    stock int    // NON exporté – uniquement dans le package boutique
}

// EXPORTÉE – accessible depuis main ou tout autre package
func NouveauProduit(nom string, prix float64) (*Produit, error) {
    if prix < 0 {
        return nil, errors.New("prix ne peut pas être négatif")
    }
    return &Produit{Nom: nom, Prix: prix, stock: 0}, nil
}

func (p *Produit) AjouterStock(qte int) { p.stock += qte }

// NON EXPORTÉE – helper interne
func (p *Produit) valider() bool {
    return p.Prix > 0 && p.Nom != ""
}
```

```
// fichier : main.go
package main
import "monprojet/boutique"

func main() {
    prod, err := boutique.NouveauProduit("Clavier", 49.99)
    if err != nil { panic(err) }
    prod.AjouterStock(100)
    // prod.stock = 5 → erreur de compilation : stock est non-exporté !
}
```

5.2 defer — garantir l'exécution d'une fonction

Analogie : `defer` = un post-it collé à la porte de sortie. Quand on quitte la fonction (`return`), Go exécute ce qui est sur le post-it. Idéal pour fermer fichiers, libérer ressources, déverrouiller mutex — **même si une erreur survient**.

```
// defer : exécuté juste avant le return, dans l'ordre LIFO (dernier entré, premier sorti)
func lireFichier(chemin string) (string, error) {
    f, err := os.Open(chemin)
    if err != nil { return "", err }
    defer f.Close() // GARANTI même si la suite plante

    // ... lire le contenu
    return contenu, nil
}
```

```
// Ordre LIFO
func exemple() {
    fmt.Println("début")
    defer fmt.Println("3 - dernier defer, premier exécuté")
    defer fmt.Println("2")
    defer fmt.Println("1 - premier defer, dernier exécuté")
    fmt.Println("fin")
}

// Sortie : début → fin → 1 → 2 → 3

// Cas classique : sync.Mutex
func (a *Annuaire) Ajouter(c Contact) {
    a.mu.Lock()
    defer a.mu.Unlock() // déverrouillage garanti
    a.contacts[c.Telephone] = c
}
```

5.3 Packages standard incontournables

```
// fmt – formatage et affichage
fmt.Println, fmt.Printf, fmt.Sprintf, fmt.Errorf

// strings – manipulation de chaînes
strings.Contains("hello go", "go") // true
strings.ToUpper("hello")           // "HELLO"
strings.Split("a,b,c", ",")        // ["a","b","c"]
strings.TrimSpace(" hello ")       // "hello"
strings.HasPrefix("golang", "go")  // true

// strconv – conversions string ↔ types
strconv.Itoa(42)                    // "42"
strconv.Atoi("123")                 // 123, nil
strconv.ParseFloat("3.14", 64)      // 3.14, nil

// math – fonctions mathématiques
math.Abs(-5.0) // 5.0
math.Sqrt(16)  // 4.0
math.Pow(2, 10) // 1024.0
math.Max(3.0, 5.0) // 5.0

// sort – tri
nums := []int{3, 1, 4, 1, 5, 9}
sort.Ints(nums) // tri sur place
strs := []string{"b", "a", "c"}
sort.Strings(strs)

// time – date et heure
now := time.Now()
fmt.Println(now.Format("02/01/2006 15:04:05"))
duration := 5 * time.Second
time.Sleep(duration)
```

Carte de révision — Jour 1 (complet)

Go = Google 2009 (Rob Pike + Ken Thompson) ; utilisé par Docker, K8s, Terraform. 25 mots-clés seulement → compilation rapide, garbage collector, concurrence native.

- **Types** : `int`, `float64`, `string`, `bool`, `byte`, `rune` — valeur zéro toujours définie
- **Variables** : `var x T = v` | `var x = v` | `x := v` (dans les fonctions !)
- **Constantes** : `const` + `iota` pour énumérations
- **Fonctions** : `func nom(a, b T) (T, error)` — retours multiples & erreur
- **Closures** : `func externe() func() T` — capture l'environnement
- **for** : seule boucle (`for`, `for cond`, `for {}`, `for i, v := range`)
- **switch** : pas de `break`, cases multiples, `fallthrough` pour enchaîner
- **Structs** : `type Nom struct {...}`, value vs pointer receiver
- **Embedding** : `type B struct { A }` — promotion méthodes/champs

- **Slice** : `[]T` dynamique | `append` / `copy` / `range` | partage l'array
- **Map** : `map[K]V` | `make()` obligatoire | `val, ok := m[k]`
- **Pointeurs** : `&` adresse, `*` déréférence, pas d'arithmétique
- **Packages** : Majuscule = exporté, minuscule = privé
- **defer** : exécuté avant return, LIFO, garanti même si erreur

Exercice noté TP1 — TechShop — Boutique CLI (45 min autonome + rendu GitHub)

Vous êtes développeur chez TechShop, une boutique de matériel informatique. Créez un gestionnaire de catalogue en ligne de commande.

Données - `Struct Produit` : `ID int`, `Nom string`, `Marque string`, `Prix float64`, `Stock int`, `Categorie string`, `Actif bool` - `Struct Catalogue` avec une slice de `Produit`

Fonctionnalités (méthodes sur Catalogue) - `AjouterProduit(p Produit) error` — erreur si ID dupliqué - `TrouverParID(id int) (Produit, error)` - `TrouverParCategorie(cat string) []Produit` - `AppliquerReduction(categorie string, pct float64) int` — retourne le nb de produits modifiés - `Vendre(id int, qte int) error` — réduit le stock, erreur si insuffisant - `Rapport() string` — résumé : nb produits, valeur totale du stock

Main - Menu CLI interactif : `[1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter` - Pré-remplir 5 produits high-tech réalistes (iPhone, MacBook...) - Gérer toutes les erreurs proprement

Rendu : fichier `tp1_techshop.go` (tout dans un seul fichier), sur un **repo GitHub dédié** (URL à envoyer dans Teams).

Critères : /5 structs + méthodes (value vs pointer) · /5 gestion d'erreurs · /5 menu CLI · /5 code propre.

Indices : `fmt.Scan(&choix)` | `for i, p := range c.produits { if p.ID == id { ... } }` | `strings.EqualFold` pour comparaison insensible à la casse.